

QR Code Based Personal Information Sharing Application

Constantin Tonu

June 22, 2026

Contents

1	Introduction	1
2	Data Format	1
2.1	Example of Shared Business Information	1
3	System Architecture and Design	2
3.1	Direct Data Exchange Through Secured QR Codes	3
3.1.1	Data Transfer Protocol	3
3.1.2	Managing Multiple Transfers	3
3.2	Data Transfer Through Secured Peer-to-Peer Connection	4
3.2.1	Data Transfer Protocol	4
3.2.2	Managing Multiple Transfers	4
4	Security Design	5
4.1	Data Privacy and Protection	5
4.1.1	Encryption at Rest	5
4.1.2	MFA Authentication Support	5
4.1.3	Shared Data Encryption	5
4.2	Secure Peer-to-Peer Connection	6
4.3	DevSecOps	6
4.4	Threats and Security Considerations	6
5	Conclusion and Future Work	7

1. Introduction

The increasing need for secure digital identity exchange and privacy-preserving information sharing has created demand for solutions that minimize reliance on centralized service providers. Traditional approaches often require users to trust third-party platforms with sensitive personal or business information, introducing privacy, security, and availability concerns.

This project proposes a decentralized system for sharing personal and business information using QR codes, asymmetric cryptography, and direct peer-to-peer communication. The objective is to enable users to exchange sensitive information securely while maintaining full control over their data. The system supports both small data transfers embedded directly into QR codes and larger transfers performed through encrypted peer-to-peer channels.

The proposed approach combines ease of use with strong security guarantees. QR codes provide a familiar user interaction model, while asymmetric encryption ensures that shared information can only be accessed by the intended recipient.

The following chapter introduces the data format used by the system and the types of information that can be exchanged.

2. Data Format

The data being transferred is structured data, similar to the information encoded by a vCard at its base, and is compatible with a form-based format. The data is organized as key-value pairs using the JSON format. This allows the system to represent simple contact information, business data, or other user-defined fields in a flexible way. The use of a vCard-like structure is relevant because vCard is a known format for representing and exchanging contact information, while JSON offers a lightweight and practical representation for mobile applications [1, 2].

The system supports sharing encrypted personal information using asymmetric keys. The shared information can be contact information similar to a vCard, additional sensitive personal information, company information if the user is a business, and also larger files, such as personal or business documents.

2.1. Example of Shared Business Information

```
{
  "company": "Acme Solutions",
  "owner": "John Smith",
  "phone": "+1-555-123-4567",
  "email": "john@acmesolutions.com",
  "website": "https://acmesolutions.com",
  "address": "123 Main Street, New York, NY"
}
```

Examples of larger files include digital scans of personal or business documents, contracts,

NDA, and financial information. When the shared information is only a few kilobytes in size, it can be embedded directly into a QR code, encrypted, and shared with the intended receiver.

If the data is too large to fit into a QR code, or if a secure peer-to-peer connection is preferred for the data transfer, the data is transferred over a secure peer-to-peer channel, also encrypted.

3. System Architecture and Design

The proposed architecture of the information sharing system is composed of a sender application and a receiver application, in a peer-to-peer communication model. The sender and the receiver can transfer information directly, without an intermediary entity, using one of two possible modes: direct data exchange through secured QR codes, or data exchange through a secured peer-to-peer connection where QR codes are used only for setting up the secure connection.

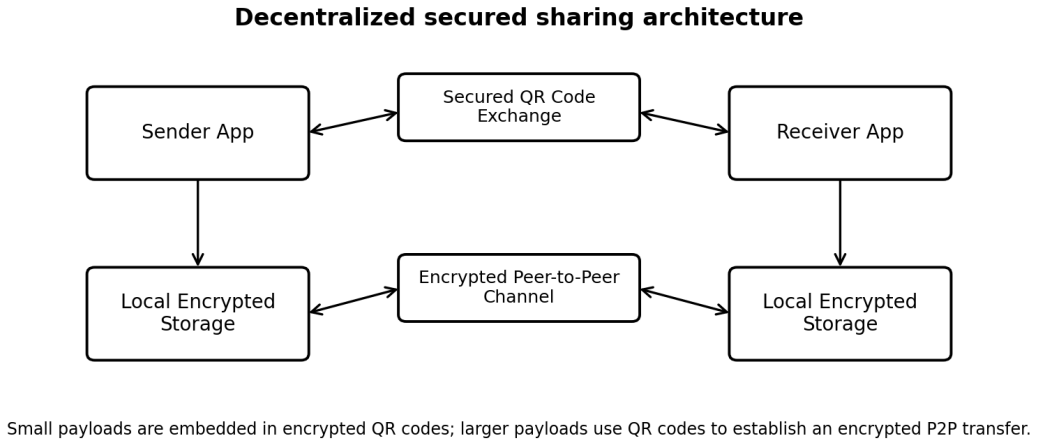


Figure 1: High-level architecture of the decentralized information sharing system.

By exchanging information and encryption keys embedded in QR codes generated on the spot, the sender app can share sensitive information stored locally on the device directly with the receiver app. For both transfer modes, data storage is decentralized and remains on the sender’s device. The receiver app obtains the requested information only after the sender reviews the request and approves which data will be shared.

The data is either embedded directly in the sender’s QR code, if it is small in size, or transferred over a peer-to-peer connection established between the sender and the receiver. In the second case, the QR code is used to exchange the information required to set up the secure connection.

3.1. Direct Data Exchange Through Secured QR Codes

In this approach, the client app creates a QR code which embeds all the information that the sender wants to share, encrypted. A lower-security mode can also be permitted where the information in the QR code is not encrypted, if the user explicitly chooses this option. However, for sensitive personal or business information, encryption should remain the default mode.

3.1.1. Data Transfer Protocol

The data transfer protocol is composed of three or four steps, depending on whether the sender or the receiver initiates the transfer.

Step 0 (optional): Sender offers to share arbitrary data with a receiver. If the data transfer is initiated by the sender, the sender creates the data package, for example a JSON-based form, and generates a QR code containing the metadata of the package. This metadata describes what the package includes and may also contain information about who the sender claims to be.

Step 1: The receiver requests specific data from a sender. The receiver selects in the app the information it needs and encodes this request as metadata in a QR code, together with information about who the receiver is. When generating the QR code, a new asymmetric key pair is generated transparently by the app, and the public key is appended automatically to the QR code.

The sender uses the receiver's public key to encrypt the data inside the sender QR code, so that only the receiver who owns the matching private key can decrypt the shared data. The receiver does not directly manage the private key, because it is handled internally by the app and is destroyed automatically after the receiver obtains and decrypts the requested information.

Step 2: The sender scans the receiver's QR code and generates the encrypted response QR code. The sender app scans the receiver's QR code using the device camera. The app decodes the request, displays to the sender the requested data and the receiver identity information, and temporarily stores the receiver public key.

The sender checks who the receiver claims to be, reviews the requested data, and selects which data will be shared. Some requested fields may be marked by the receiver as optional. If the sender agrees to share data, the sender app generates a QR code containing the selected data encrypted with the receiver's public key.

Step 3: The receiver scans the sender's QR code and receives the data. The receiver scans the sender's QR code using the receiver app. The app decodes the QR code, decrypts the data using the matching private key, saves the transferred data locally, and automatically deletes the temporary key pair after the transfer is completed.

3.1.2. Managing Multiple Transfers

Because the protocol is asynchronous and is based on scanning multiple QR codes, a session-like transfer identifier is embedded in the QR code in Step 0 or Step 1. This identifier is reused in the following QR codes by both the sender app and the receiver

app, allowing both sides to keep track of different initiated transfers.

3.2. Data Transfer Through Secured Peer-to-Peer Connection

This proposed data transfer method avoids embedding the data directly into the QR code itself. It is also needed when the system must transfer more data than can be contained in a single QR code, for example digital copies of documents or larger business files.

Step 0 and Step 1 remain the same as in the direct QR-code-based transfer method. The only changes are in Step 2, where the sender QR code contains connection details instead of the full data package, and Step 3, where the receiver uses those details to retrieve the encrypted data package over a secure peer-to-peer connection.

3.2.1. Data Transfer Protocol

Step 0 (optional): Sender offers to share arbitrary data with a receiver. This step works the same as in the direct QR-based method.

Step 1: The receiver requests specific data from a sender. This step is also identical to the first transfer method described in Section 3.1.1.

Step 2: The sender prepares the encrypted data package and generates the connection QR code. To establish the secured connection, the sender QR code embeds the URI of the sender's temporary data transfer service, which is hosted locally on the sender's device through the sender app. At that location, the sender app stores the requested data package, encrypted with the receiver's public key.

Besides the URI, the parameters needed to secure the connection are also embedded in the QR code. This information is encrypted with the receiver's public key, so that only the intended receiver can decrypt the URI and connection parameters.

Step 3: The receiver scans the sender's QR code and receives the data. The receiver scans the sender QR code through the receiver app. The app automatically decodes and decrypts the connection information using the matching private key. It extracts the URI and the secure connection parameters, connects to the sender device through the peer-to-peer connection, downloads the encrypted package, decrypts it locally, saves the data, and deletes the temporary key pair.

An authentication step can be used to enforce secure authentication before the package can be downloaded. In this case, the receiver proves that it is the intended entity by digitally signing a challenge provided at the URI, using the same private key as before. If the signature is verified correctly by the sender using the receiver's public key, the download can be permitted.

3.2.2. Managing Multiple Transfers

Unlike the direct QR-code-based transfer method described in Section 3.1.2, support for managing multiple concurrent transfer sessions is not included in the first iteration of the peer-to-peer transfer mechanism. The peer-to-peer transfer protocol is currently designed to handle a single transfer session at a time. This design decision reduces implementation complexity during the proof-of-concept phase and allows the security architecture of the

protocol to be validated before introducing additional session management mechanisms.

4. Security Design

Being a system designed specifically for easily sharing sensitive personal and business information over QR codes, the security posture of the system needs to be considered from the design phase, following the recommended practices of a secure-by-design architecture.

The security concerns span the system components, communication protocols, encryption methods, data privacy, data protection, and other aspects. This section discusses the main security concerns of the system and will be extended in future reports.

4.1. Data Privacy and Protection

Because the system mainly deals with sensitive information of physical persons and companies, stored locally on the device, embedded in QR codes, and transferred over network connections, ensuring the protection and privacy of the data is crucial.

4.1.1. Encryption at Rest

Data is stored locally on the device of the data owner, encrypted. The encryption key is managed by the application and is protected through a PIN, password, or biometric authentication. This reduces the risk that locally stored data can be accessed if the device is lost or stolen.

4.1.2. MFA Authentication Support

The authentication mechanism will support username and password login, biometric authentication, and third-party authentication. Multi-factor authentication is also supported for stronger security consolidation of the system. This is aligned with modern digital identity guidance, where authentication assurance and protection of authenticators are important security requirements [7].

4.1.3. Shared Data Encryption

The data being shared is encrypted using asymmetric keys owned by the receiver app. Each time new data is shared from the sender to the receiver, the receiver app transparently creates a new asymmetric key pair, stores internally the private key for data decryption, and shares the public key with the sender over a QR code.

The sender encrypts the shared data using the receiver's public key. After the receiver successfully obtains the encrypted data, the receiver app decrypts it automatically using the private key, saves the data locally, and immediately deletes the temporary key pair.

The immediate deletion of the key pair by the receiver app is intended to reduce the risk of an attacker stealing private keys from the receiver and later decrypting a leaked QR code.

4.2. Secure Peer-to-Peer Connection

For large files or data transfers larger than a few kilobytes, which cannot be embedded directly into a QR code, a secure peer-to-peer connection is established between the sender and the receiver. The sender prepares the encrypted package and hosts a temporary file sharing service through the sender app.

The receiver connects to the sender app through a secure connection and may solve a cryptographic challenge provided by the sender, using the same key pair, to prove its identity. After successful verification, the receiver is permitted to download the data.

The security parameters of the connection and the sender temporary URI are embedded in the QR code generated by the sender. These details are encrypted using the receiver's public key, so that only the intended receiver can decrypt the URI and the connection security parameters. This prevents unauthorized actors who may obtain the sender QR code from connecting to the sender app and downloading the package.

4.3. DevSecOps

The development of the app will use a secure CI/CD pipeline to further increase the security posture of the system. The pipeline can include dependency scanning, static application security testing, secret detection, container scanning if applicable, and automated security checks before release.

4.4. Threats and Security Considerations

Several threats need to be considered when designing this system. A QR code may be intercepted by an attacker through a picture or screen capture. For this reason, sensitive payloads and connection parameters should be encrypted by default. Malicious QR code replacement is another relevant risk, where an attacker displays a forged QR code instead of the legitimate one. Future versions of the protocol may reduce this risk by adding digital signatures and stronger sender and receiver identity verification.

Replay attacks should also be considered. A previously captured QR code could be reused by an attacker if the protocol does not include expiration timestamps or session identifiers. Therefore, transfer sessions should be short-lived and linked to unique session identifiers.

The current design assumes that the sender and receiver endpoints are trusted and uncompromised. While the proposed architecture incorporates multiple security controls, including asymmetric encryption, temporary key generation, encrypted local storage, and optional challenge-response authentication, a formal threat model has not yet been completed. Developing a comprehensive threat model and performing a systematic security analysis will be part of future work and will help identify additional attack vectors, trust assumptions, and mitigation strategies.

Other risks include private key theft, device compromise, unauthorized local access, and metadata leakage. The temporary key lifecycle reduces the exposure of cryptographic material, but it does not protect against malware running on the endpoint device. For this reason, endpoint security, secure storage, and careful permission management remain important parts of the overall system security.

5. Conclusion and Future Work

This report presented the high-level architecture, communication protocols, and security foundations of the proposed decentralized information-sharing system. The current design demonstrates the feasibility of secure QR-code-based and peer-to-peer information exchange while preserving user privacy and data ownership.

In the next iteration, the main focus will be on improving and consolidating the security architecture of the system, as well as developing a proof of concept for the sender and receiver apps.

The legal and security concerns related to different regulations and security frameworks, such as GDPR and NIST guidance, will also have a dedicated chapter in the next report. The purpose of this chapter will be to identify and document the security risks and compliance aspects that may affect the system.

A few practical use cases for each of the two proposed transfer methods will also be presented. The analysis will focus on the security provided by the system, but also on its weaknesses, risks, identified security issues, and possible prevention and mitigation actions.

References

- [1] S. Perreault, “vCard Format Specification,” RFC 6350, Internet Engineering Task Force, Aug. 2011.
- [2] T. Bray, “The JavaScript Object Notation (JSON) Data Interchange Format,” RFC 8259, Internet Engineering Task Force, Dec. 2017.
- [3] D. Koradia and V. Agrawal, “Self-Sovereign Identity Management System Incorporating Blockchain,” 2024.
- [4] M. Babel et al., “Self-Sovereign Identity and Digital Wallets,” *Electronic Markets*, Springer, 2025.
- [5] M. Schanzenbach, G. Bramm, and J. Schütte, “reclaimID: Secure, Self-Sovereign Identities using Name Systems and Attribute-Based Encryption,” arXiv:1805.06253, 2018.
- [6] M. Khan et al., “Blockchain-Based Identity Management Systems: A Review,” 2020.
- [7] National Institute of Standards and Technology, “Digital Identity Guidelines,” NIST Special Publication 800-63-4, 2025.
- [8] IDnow, “Digital Identity Wallet Platform Documentation,” 2025.
- [9] Evrotrust, “Digital Identity and Trust Services Platform Documentation,” 2024.